

# Holon: Agents that act and follow through

## Create and manage AI agents with ongoing responsibilities.

Give an agent work such as sorting incoming mail, following up on order issues, investigating problems, or reviewing code. You set the goal and requirements; it uses tools to do the work, saves progress while waiting, and continues when the conditions are met.

## Act on changes and schedules

Once connected, new emails, order updates, or an agreed time can prompt the agent to act.

## Keep progress and follow through

Goals, findings, and next steps stay with the work. Replies and new results let it pick up where it left off.

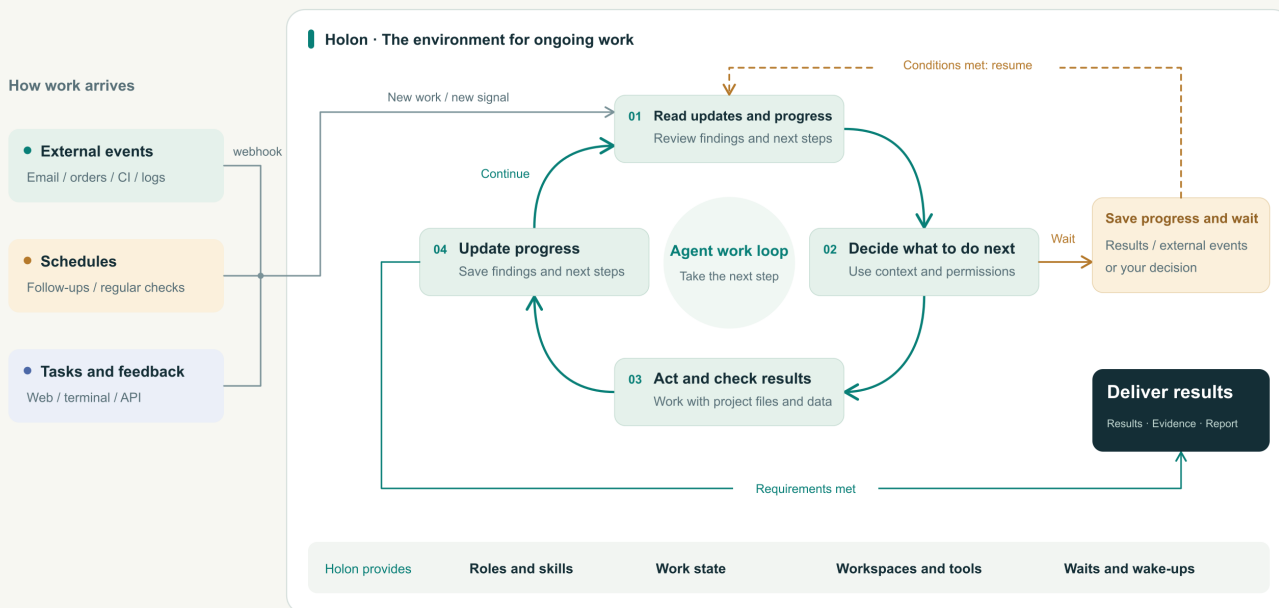
## Check in and guide the work

See progress, add context, or change direction. When a decision needs your judgment, the agent asks you.

## Respond to change. Keep work moving.

HOLON / WORK MODEL

The agent decides and acts. Holon preserves progress, manages the environment, and wakes it when needed.



Continue when ready. Wait for replies or results. Deliver when requirements are met.

Draft illustration · 2026-09-18

# What Holon provides

## Act on changes and schedules

| Feature                         | What it does                                                                 | Why it helps                                                                 |
|---------------------------------|------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| <b>External events</b>          | Receives messages through Webhooks, or signals an agent to check for updates | Mail and order changes can start work without someone forwarding each update |
| <b>Scheduled tasks</b>          | Runs at an agreed time or checks at regular intervals                        | Follows up, monitors, and summarizes without repeated reminders              |
| <b>Progress-aware follow-up</b> | Reads its role, findings, and next steps when a message arrives              | Continues the same work without starting the explanation over                |

Integration depends on the source system. Webhook support allows direct notifications; reading mail, filtering content, or converting formats requires suitable tools or adapters. The agent then considers the change alongside existing progress.

## Reuse roles and working methods

| Feature                  | What it does                                                                                     | Why it helps                                               |
|--------------------------|--------------------------------------------------------------------------------------------------|------------------------------------------------------------|
| <b>Persistent agents</b> | Keeps each agent's role, memory, and work records                                                | Supports ongoing responsibilities with accumulated context |
| <b>Agent templates</b>   | Creates agents from role instructions and skill settings; templates can be customized and shared | Reuses a role without configuring it from scratch          |
| <b>Skills per agent</b>  | Manages reusable methods and enables or disables them for each agent, to be read as needed       | Gives each role its own methods, adjustable independently  |
| <b>Model choice</b>      | Sets a default model, with optional choices per agent                                            | Matches the model and provider to the task                 |

For example, a template can define a support role, skills can explain how to classify and answer requests, and "follow up on this return request" is one work item. **Templates define roles; skills describe methods; work items record goals and progress.** This is a configuration example, requiring the relevant business integrations.

## Keep progress while waiting; leave evidence on completion

| Feature                        | What it does                                                                                         | Why it helps                                                            |
|--------------------------------|------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| <b>Work items</b>              | Records each task's goal, plan, checklist, waiting reasons, and completion report                    | Shows what is done, what is holding it up, and what counts as complete  |
| <b>Wait and resume</b>         | Tracks pending execution results, replies, and external conditions                                   | Continues the same work when those conditions are met                   |
| <b>Delegation and tracking</b> | Assigns tasks to subagents, tracks progress and results, and allows additional input or cancellation | Keeps ownership, progress, and results visible                          |
| <b>Inspectable results</b>     | Preserves message sources, tool activity, and completion reports                                     | Lets you read the conclusion first and inspect the evidence when needed |

## Organize the working environment

| Feature                          | What it does                                                                     | Why it helps                                                                 |
|----------------------------------|----------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| <b>Workspaces</b>                | Lets one agent use multiple workspaces and switch between them                   | Works across project files and directories toward one goal                   |
| <b>Separate code directories</b> | Uses Git worktree to create independent coding directories                       | Supports parallel changes, experiments, and reviews                          |
| <b>Web and terminal</b>          | Connects Web and TUI interfaces to one service, with CLI and HTTP API access too | Lets you return to the same agents and work through your preferred interface |
| <b>Self-hosting</b>              | Bundles the Web interface with the binary for a personal computer or server      | Needs no separate frontend deployment and can serve a remote team            |

Choose who owns the work, which files and tools they use, and how to inspect progress and results.

# A reviewer that follows through

For individual developers and teams that need ongoing code review.

Create a review agent from a template, configure skills, a project workspace, and GitHub events, then give it an ongoing responsibility:

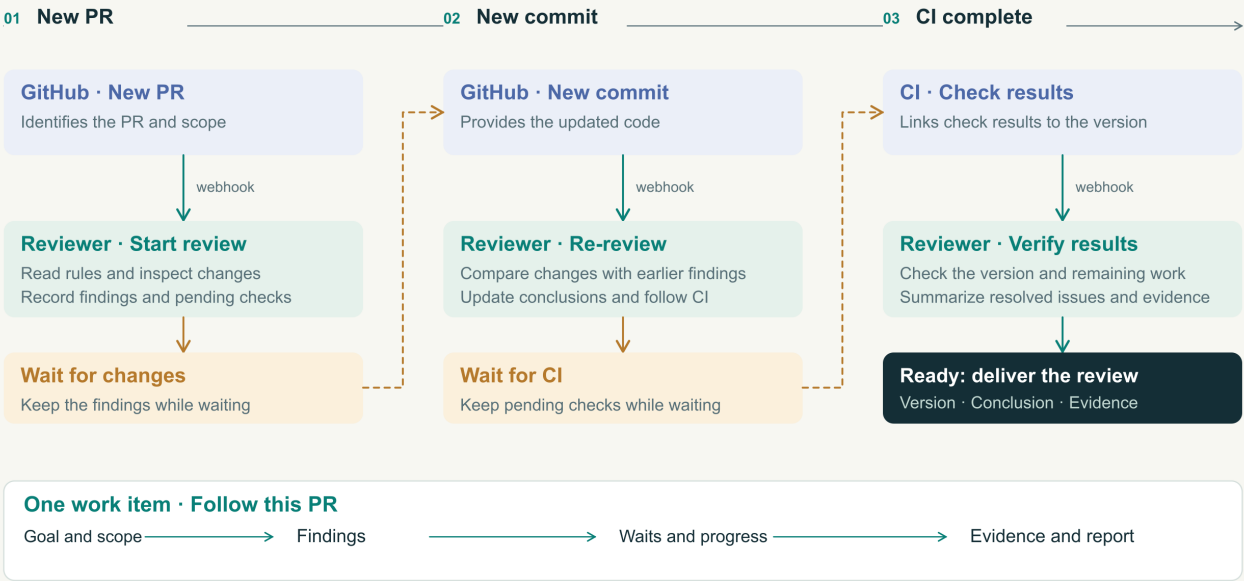
Review this repository's pull requests (PRs), focusing on compatibility and regressions. Follow up when code changes or automated checks (CI) finish. Ask me about disputed trade-offs. Keep a conclusion and supporting evidence for each PR.

For example, CI can send its result and the relevant PR through a Webhook. On failure, the agent investigates; on success, it checks for any remaining issues.

## One PR. Review as new updates arrive.

HOLON / CONTINUOUS REVIEW

New code or check results prompt the reviewer to revisit its earlier findings.



Shown with issues resolved. Open issues prompt follow-up; disputed choices wait for your decision.

Draft illustration · 2026-09-18

**Each PR has its own work record.** When revised code arrives, the agent checks it against earlier findings. It follows up on unresolved issues and delivers a report once the requirements are met.

Close the terminal and return later through a browser. While the host and Holon's background service remain running, the agent can keep receiving updates and following up.

Illustrative workflow. Automatic GitHub updates require tools, account permissions, and event integration. You can also supply new results manually. Publishing or merging code requires authorization.

# Give shared agents a role in the team

Run Holon on a team server. Shared agents can own investigation, review, or acceptance testing. Members use a browser or terminal to see the same progress, add context, or take over.

A four-person AI hardware team's experience illustrates two handoffs:

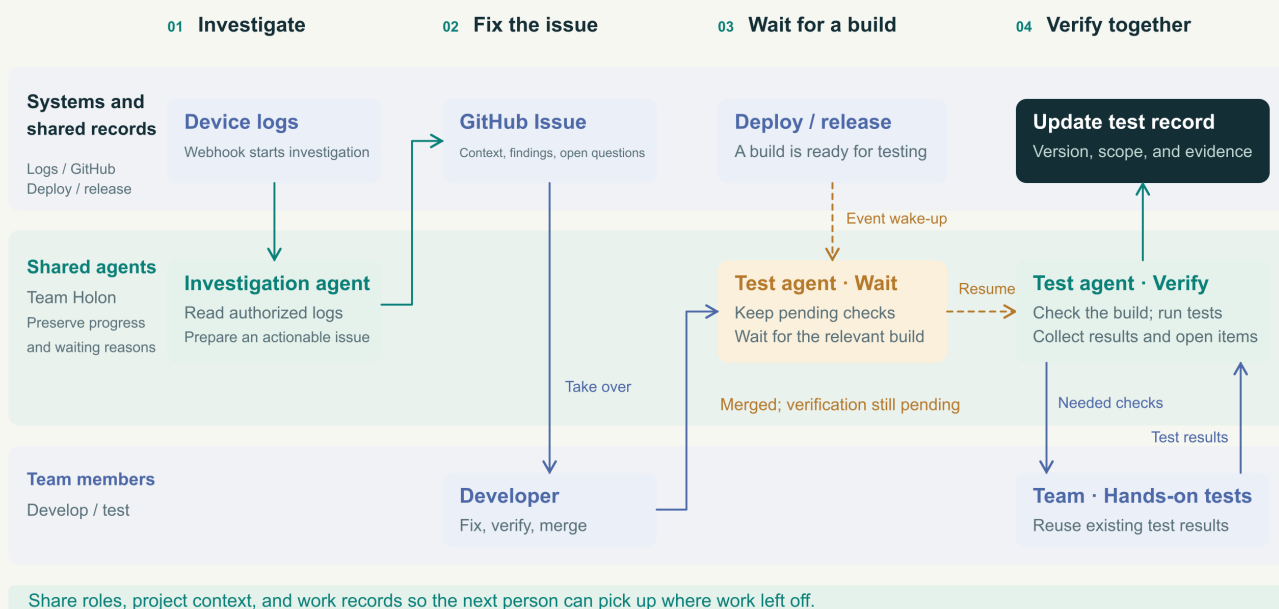
**Investigation agent: turn field evidence into an actionable issue.** It organizes device logs and findings into an issue that a developer can work on.

**Testing agent: follow a fix through to verification.** After code merges, it keeps the pending checks. Once a build containing the fix is testable, it runs automated checks and brings together the team's hands-on results.

## Shared agents keep handoffs moving.

HOLON / TEAM HANDOFF

From investigation to verification, each handoff keeps progress and results visible.



Based on two team handoffs. Check which fixes a testable build contains; keep following up on the rest.

Draft illustration · 2026-09-18

**The team can see who is following up, what has changed, and what is needed next.** Findings and next steps survive a handoff. Work waiting for a deployment or reply resumes when results arrive.

Scheduled checks can also identify which fixes are ready to test, which are still waiting, and which need someone to retest them.

The diagram combines two handoffs from the team's account, not one measured end-to-end run. Scheduled checks are a configuration example. Logs, GitHub, and release systems require integration and access permissions.

# Put your first agent to work

Start with one concrete task, then add events and schedules as needed.

- 1. Create from a template.** Choose an Agent Template to reuse a role and its working methods.
- 2. Define responsibilities.** State what it owns, what to deliver, and when to ask you. Choose skills and the files and systems it may access.
- 3. Assign a task.** Provide the goal, materials, and completion criteria. Review progress, add context, and make decisions when needed.
- 4. Connect updates and schedules.** Receive business messages through Webhooks or adapters, or schedule regular follow-ups.

The screenshot displays the Holon web interface. On the left, a sidebar contains navigation links: Dashboard, Search, Skills, Agent Templates, and Settings. Below these, a section titled 'AGENTS 3' shows a list of agents: 'INV Investigation' (investigator), 'REV Code review' (reviewer), and 'TES Verification' (tester). The 'REV Code review' agent is selected. The central workspace shows a 'Code review' task. It includes an 'External message' from a GitHub webhook about a new commit on PR #42. The task status is 'Completed'. The workspace contains text updates: 'Changes reviewed; waiting for CI', 'The skipped-page issue is fixed. The latest commit adds a regression test for the final page.', and a 'Completed' section with bullet points: 'Checked the pagination change against earlier findings.' and 'Confirmed tests cover the final page and empty lists.' Below this, a 'Next step' section states: 'Resume when CI results arrive. Summarize the review if checks pass; investigate if they fail. Findings, check progress, and waiting conditions remain in the same work item.' A timestamp '10:15 AM' is shown. At the bottom, there is a 'Message reviewer...' input field and a 'review-model' dropdown. The right sidebar, titled 'Work item detail', shows tabs for Overview, Files, and Details. The 'Details' tab is active, displaying 'Follow PR #42: fix skipped pages' with a 'Blocked' status and 'Current' priority. It includes an update timestamp 'Updated · 9/17/2026, 10:15:00 AM'. A 'Blocked by' section explains: 'Waiting for CI on the current commit; resume when results arrive.' Below this, a 'Plan' section lists tasks: 'work-items/demo-reviewer-work/plan.md', 'Verify the pagination fix without breaking existing clients.', and 'Check the changes against earlier findings, verify CI for this commit, and summarize the review.' A 'Todo' section shows progress: 'Check pagination and compatibility' (Completed), 'Review the fix and new tests' (Completed), 'Check CI for the current commit' (Pending), and 'Summarize conclusions and evidence' (Pending). A 'Technical details' section is partially visible at the bottom.

Workbench shown with demo data. Choose an agent on the left, read updates in the center, inspect the plan and waiting reason on the right, and add input below. A terminal interface connects to the same service.

Run Holon on your computer or a team server; the Web interface is included. Configure a model, tools, and workspace. Keep the host and background service running for ongoing follow-up.

**Get started:** [Installation and quick start](#) · [Source code](#) · [Documentation](#)

**Questions and feedback:** [hello@holon.run](mailto:hello@holon.run)